

Computer Networks Assignment

IP Addressing, Routing, Congestion Control and Fragmentation

Name: Gurkirat Singh
Bhangoo
Roll No.: 24bai70370
Course: Computer Networks
Date: 26/9/2026

Q1. IP Addressing, IPv4 Classes, Subnetting and Supernetting

1. What is an IP Address?

An Internet Protocol (IP) address is a logical numerical address assigned to a device or network interface participating in an IP network. It is used to identify the source and destination of packets so that data can be delivered from one network to another. In IPv4, an address is 32 bits long and is normally written as four 8-bit decimal numbers separated by dots, for example, 192.168.1.10.

2. Why were classes needed in IPv4?

The original IPv4 addressing system used classes to divide the 32-bit address space into predefined network and host portions. This made address allocation and routing simpler because organizations could receive a block according to the approximate size of their network. The five traditional classes were A, B, C, D and E. Classes A, B and C were used for unicast networks, Class D for multicast, and Class E was reserved for experimental use.

Classful addressing was later replaced in practical Internet routing by Classless Inter-Domain Routing (CIDR), because fixed classes could waste large numbers of addresses. Nevertheless, the class system is important for understanding the fundamentals of IPv4 addressing.

3. IPv4 Address Classes

Class	First Octet	Default Mask	Network Bits	Possible Networks	Hosts per Network
A	1–126	255.0.0.0 (/8)	8	126	16,777,214
B	128–191	255.255.0.0 (/16)	16	16,384	65,534
C	192–223	255.255.255.0 (/24)	24	2,097,152	254
D	224–239	Multicast	—	—	—
E	240–255	Experimental/Reserved	—	—	—

Note: The Class A network range 1–126 gives 126 usable classful network numbers because 0.0.0.0/8 is reserved and 127.0.0.0/8 is reserved for loopback. The host counts for Classes A, B and C are calculated as $2^{\text{host_bits}} - 2$, excluding the network address and broadcast address.

4. Understanding the Classes

- Class A: Designed for very large networks. It uses 8 bits for the network portion and 24 bits for hosts.
- Class B: Designed for medium-sized networks. It uses 16 bits for the network portion and 16 bits for hosts.
- Class C: Designed for smaller networks. It uses 24 bits for the network portion and 8 bits for hosts.
- Class D: Used for multicast communication rather than ordinary network/host allocation.
- Class E: Reserved for experimental purposes.

5. What is Subnetting?

Subnetting is the process of dividing one larger IP network into multiple smaller logical networks called subnets. It is done by borrowing bits from the host portion of an address and using them as additional network/subnet bits. Subnetting improves address utilization, reduces broadcast traffic and makes network management easier.

Example: Suppose we have the Class C network 192.168.1.0/24 and want four equal subnets.

- Original network: 192.168.1.0/24
- Borrow 2 host bits because $2^2 = 4$ subnets.
- New prefix: /26
- New subnet mask: 255.255.255.192
- Each subnet contains 64 addresses, of which 62 are normally usable host addresses.

Subnet	Network Address	Usable Host Range	Broadcast
1	192.168.1.0/26	192.168.1.1 – 192.168.1.62	192.168.1.63
2	192.168.1.64/26	192.168.1.65 – 192.168.1.126	192.168.1.127
3	192.168.1.128/26	192.168.1.129 – 192.168.1.190	192.168.1.191
4	192.168.1.192/26	192.168.1.193 – 192.168.1.254	192.168.1.255

6. What is Supernetting?

Supernetting is the opposite of subnetting. It combines multiple smaller, contiguous networks into one larger logical address block. It is commonly associated with CIDR and route aggregation, where several routes can be represented by one summarized route. This reduces the size of routing tables.

Example: Four consecutive /24 networks—192.168.0.0/24, 192.168.1.0/24, 192.168.2.0/24 and 192.168.3.0/24—can be summarized as 192.168.0.0/22, provided the networks are properly aligned and contiguous.

Thus, instead of advertising four separate routes, a router can advertise one summarized route: 192.168.0.0/22.

Q2. Link-State and Distance-Vector Routing

1. Link-State Routing

In link-state routing, each router builds a detailed map of the network topology. A router discovers its directly connected neighbors and the cost of each link, then distributes this information through link-state advertisements. Each router uses the collected information to construct the same network topology database and runs a shortest-path algorithm, typically Dijkstra's algorithm, to calculate the least-cost paths.

Example:

- Suppose A is connected to B with cost 2 and C with cost 5.
- B is connected to C with cost 1.
- The direct path $A \rightarrow C$ costs 5.
- The path $A \rightarrow B \rightarrow C$ costs $2 + 1 = 3$.
- Dijkstra's algorithm therefore selects $A \rightarrow B \rightarrow C$ as the least-cost path.

2. Distance-Vector Routing

In distance-vector routing, a router does not maintain a complete map of the network. Instead, it maintains a table containing the best known distance (cost) to each destination and the next hop used to reach it. Routers exchange these distance vectors with neighboring routers and repeatedly update their tables using the Bellman-Ford principle until the routes converge.

Example using the same network:

- A initially knows B at cost 2 and C at cost 5.
- B tells A that it can reach C with cost 1.
- A calculates the cost through B as $\text{cost}(A,B) + \text{cost}(B,C) = 2 + 1 = 3$.
- Since 3 is less than the direct cost of 5, A updates its route to C to use B.
- Therefore, A selects $A \rightarrow B \rightarrow C$ with total cost 3.

3. Difference Between Link-State and Distance-Vector Routing

Feature	Link-State	Distance-Vector
Network knowledge	Maintains a detailed topology map.	Maintains distances and next hops.
Information exchanged	Link-state information about links/neighbors.	Distance vectors/routing tables with neighbors.
Typical algorithm	Dijkstra's Shortest Path First.	Bellman-Ford principle.
Path calculation	Each router calculates paths from the topology database.	Routes are calculated from information received from neighbors.
Convergence	Generally faster after topology changes.	Can be slower and may experience routing loops during convergence.
Resource usage	Usually requires more memory and processing.	Generally simpler with lower resource requirements.

Both approaches aim to find a least-cost path. The key difference is how they obtain the information needed for that calculation: link-state routers build a network map and calculate shortest paths, whereas distance-vector routers learn route costs iteratively from neighboring routers.

Q3. Network Congestion, Congestion Control, Leaky Bucket, Token Bucket and Fragmentation

1. What is Congestion in a Network?

Network congestion occurs when the amount of traffic entering a network or router is greater than the network's ability to process and forward it. As queues fill up, packets experience increasing delay and may eventually be dropped when buffers become full.

2. Why Do We Need Congestion Control?

- To prevent excessive packet loss caused by overflowing buffers.
- To reduce unnecessary retransmissions and wasted bandwidth.
- To control queueing delay and improve overall network performance.
- To distribute traffic more efficiently across available network capacity.
- To maintain stable and reliable communication when traffic load increases.

3. Leaky Bucket Algorithm

The Leaky Bucket algorithm controls traffic by allowing packets to leave a buffer at a fixed rate, similar to water leaking from a bucket through a small hole. Incoming packets may arrive in bursts, but the output is smoothed to a relatively constant rate. If the bucket/buffer becomes full, additional packets may be discarded.

Example:

- Assume the bucket can hold 5 packets and transmits 1 packet per second.
- If 5 packets arrive at once, they are stored and transmitted one per second.
- If another 3 packets arrive while the bucket is already full, packets that cannot fit may be dropped.
- Thus, the output rate remains controlled rather than following the bursty input rate.

4. Token Bucket Algorithm

The Token Bucket algorithm also controls traffic, but it allows controlled bursts. Tokens are generated at a fixed rate and stored in a bucket up to a maximum capacity. A packet can be transmitted only when enough tokens are available. Because tokens can accumulate during periods of low traffic, a sender can later transmit a burst of packets while still respecting the long-term average rate.

Example:

- Suppose tokens are generated at 2 tokens per second and the bucket can store 10 tokens.
- If the network is idle for 5 seconds, up to 10 tokens can accumulate.
- A burst of packets can then be transmitted using those accumulated tokens.
- After the stored tokens are consumed, transmission again depends on newly generated tokens.

5. Difference Between Leaky Bucket and Token Bucket

Feature	Leaky Bucket	Token Bucket
Output	Usually produces a fixed/smoothed output rate.	Allows bursts when tokens have accumulated.
Burst traffic	Generally suppresses bursts.	Allows controlled bursts.
Overflow	Packets may be discarded when the bucket is full.	Packets may be rejected/delayed when insufficient tokens are available, depending on implementation.
Main purpose	Traffic shaping and smoothing.	Traffic policing/shaping while

		allowing bursts.
Basic idea	Packets leave at a controlled rate.	Packets consume tokens to gain permission to transmit.

6. Why is Fragmentation Needed?

Fragmentation is the process of dividing an IP packet into smaller pieces when the packet is larger than the Maximum Transmission Unit (MTU) supported by a network link. Different networks can have different MTU sizes, so a packet that is acceptable on one network may be too large for the next link.

For example, suppose a packet is 4000 bytes but the next network link has an MTU of 1500 bytes. Without a mechanism to handle the size difference, the packet cannot be transmitted over that link as one unit. IPv4 can fragment the packet into smaller fragments that fit within the MTU. The destination can then reassemble the fragments into the original packet.

Important points:

- Fragmentation allows packets to cross links with smaller MTUs.
- Each fragment receives information that allows the destination to identify and reassemble the fragments.
- Fragmentation creates additional processing and overhead and can reduce efficiency.
- In IPv4, routers may fragment packets when permitted by the packet's flags and the network conditions.
- IPv6 routers do not perform fragmentation; the sending host is responsible for using an appropriate packet size.

Conclusion

IPv4 addressing provides a logical way to identify devices and networks, while subnetting and supernetting improve the organization and scalability of address allocation. Link-state and distance-vector routing use different forms of network information to determine least-cost paths. Congestion control helps prevent network overload, with the Leaky Bucket and Token Bucket providing different traffic-control behaviors. Finally, fragmentation addresses differences in MTU sizes so that packets can traverse networks with smaller transmission limits.